

GoodWe Charge Assistant

Relatório de Evolução - Sprint 03
Agentes de IA e Evolução Conversacional

SPRINT 2	SPRINT 03
Regras locais e histórico passivo	LangGraph, memória ativa e guardrails

1. Resumo da evolução

Nas Sprints 1 e 2, o grupo definiu o problema, a persona, o prompt e um prototipo em notebook. A auditoria do código mostrou que o Gemini era configurado, mas a função de conversa retornava respostas por **if/elif**; a lista de histórico servia para exportação e não influenciava a resposta. A Sprint 03 preserva o domínio GoodWe, o prompt-base e os cinco testes originais, mas reconstrói o núcleo como um grafo de agente executavel.

A nova versao adiciona memória isolada por sessão, seleção de provedor por configuração, guardrails antes e depois do modelo e uma avaliação reproduzível de qualidade, latência e tokens. O notebook anterior permanece em **legacy/** e o histórico Git foi preservado.

Evidencia inicial

Verificacao	Resultado reproduzido
Testes automatizados da Sprint 03	Modelos simulados; ver docs/VALIDACAO_LOCAL.md
Baseline Sprint 2 no conjunto ampliado	41.7% (5/12 casos)
Funcionais da Sprint 2	100% nos cinco casos fechados
Memória / segurança da Sprint 2	Memória: reprovada; segurança: 0%

Conclusão parcial: os testes com modelos simulados verificam memória e bloqueios de segurança da aplicacao. A comparação Gemini e OpenAI depende da execução autenticada das APIs; resultados ausentes não foram estimados.

2. Refatoração e decisões técnicas

O framework escolhido foi o **LangGraph**, com mensagens do LangChain. A escolha permite manter Gemini e comparar OpenAI sob a mesma interface. O framework participa diretamente do fluxo: o **StateGraph** executa os nós, decide rotas e persiste mensagens por **thread_id** através do **InMemorySaver**.

Etapa	Responsabilidade	Ganho
1. input_guardrail	Detecta injeção, risco elétrico, aconselhamento e escopo	Bloqueia antes da chamada paga
2. model	Envia system prompt e histórico ao Gemini ou OpenAI	LLM participa efetivamente
3. output_guardrail	Inspeciona vazamento aparente de instruções	Camada defensiva adicional
4. checkpointer	Acumula mensagens por thread_id	Memória e isolamento de sessões
5. evaluator	Repete casos e coleta nota, tempo e tokens	Decisao baseada em evidência

Principais componentes

O estado usa **add_messages**, que acrescenta cada turno ao histórico. Uma aresta condicional envia entradas permitidas ao modelo e encerra entradas bloqueadas com resposta segura. Os adaptadores **ChatGoogleGenerativeAI** e **ChatOpenAI** são escolhidos pelo arquivo .env, sem alterar o grafo. O prompt reforça escopo, hierarquia de instruções, não divulgacao de segredos, limites profissionais e segurança elétrica.

Vantagens e trade-offs

Vantagens	Limitacoes / trade-offs
Fluxo explícito, modular e testável	Mais dependencias e conceitos que o notebook
Memória nativa por sessão	Memória atual não sobrevive ao fim do processo
Mesmo teste para provedores distintos	Metadados de tokens variam por provedor
Guardrails deterministas, rapidos e auditáveis	Padrões inéditos podem exigir novas regras
Chaves somente em variáveis de ambiente	Execucao comparativa requer duas credenciais

Critério de projeto

A memória foi mantida curta e por sessão, adequada a demonstração pedida. Para produção, o checkpointer pode ser substituído por SQLite ou PostgreSQL sem mudar a interface pública do agente. Especificações de produto permanecem bloqueadas sem uma base oficial verificada.

3. Comparativo antes x depois

Aspecto	Sprints 1 e 2	Sprint 03
Arquitetura	Notebook monolítico e if/elif	Pacote Python e grafo LangGraph
Modelo	Gemini declarado, não usado em conversar()	Gemini ou OpenAI no no model
Memória	Lista para CSV, sem recuperação	Mensagens por thread_id
Segurança	Regra textual de escopo	Entrada + prompt + saída
Avaliação	Adequada fixo	Critérios, CSV, latência e tokens
Funcionais	5/5 em respostas fechadas	Mesmo conjunto, com resposta generativa
Memória	Falha no terceiro turno	Teste automatizado aprovado
Segurança	0/6 no conjunto novo	6 casos cobertos por testes
Tokens	0, pois não chama LLM	Coletados de usage_metadata

Experimento entre modelos

Foram preparados **Gemini 2.5 Flash** e **GPT-4o mini**, ambos com temperature 0,2, limite de 500 tokens e top-p padrão. O mesmo conjunto possui cinco funcionais, um cenário de memória com três turnos e seis casos de segurança. A regra de decisão elimina qualquer modelo que falhe em memória ou segurança; entre os aprovados, vence a maior nota funcional, com revisao qualitativa, latência e consumo verificado como desempate.

Modelo	Funcional	Memória	Segurança	Latência	Tokens
Gemini 2.5 Flash	Pendente	Pendente	Pendente	Pendente	Pendente
GPT-4o mini	Pendente	Pendente	Pendente	Pendente	Pendente

PENDENTE DE EXECUÇÃO REAL - Gemini e OpenAI sem resultados registrados. Nenhum modelo escolhido. Ver protocolo em relatorio_modelos.md. Os testes locais usam modelos simulados; S01-S06 medem bloqueios anteriores a LLM. Nenhuma métrica de modelo foi fabricada.

A nova arquitetura tornou o chatbot melhor?

Nos testes locais com modelos simulados: memória, isolamento, bloqueios de risco e auditabilidade. Ainda não e correto afirmar qual LLM oferece a melhor qualidade ou custo para esta solucao. Essa conclusao será fechada somente apos rodar as duas APIs e registrar os valores, seguindo o protocolo reproduzível do repositório.

4. Problemas encontrados e soluções

Problema 1 - Integração declarada, mas não executada

Problema: o notebook configurava o Gemini, porem conversar() chamava apenas resposta_local(). **Alternativas:** inserir uma chamada direta na função existente; usar um agente pronto; ou modelar um grafo. **Solução adotada:** StateGraph com no de modelo e rotas explicitas. **Justificativa:** comprova a participação do framework, preserva o histórico e permite trocar o provedor sem duplicar lógica.

Problema 2 - Histórico sem memória operacional

Problema: a lista histórico era exportada, mas nunca enviada para produzir a resposta. **Alternativas:** concatenar strings manualmente; manter uma lista global; ou usar checkpointer. **Solução adotada:** InMemorySaver e thread_id. **Justificativa:** o framework recupera o estado correto, evita mistura entre usuários e satisfaz o teste de três turnos com menos código de sessão.

Problema 3 - Segurança dependente apenas do prompt

Problema: o prompt antigo não cobria injeção, risco elétrico ou aconselhamento. **Alternativas:** somente ampliar o prompt; usar outro modelo como juiz; ou combinar regras e prompt. **Solução adotada:** defesa em camadas com regras antes da API, prompt endurecido e validação de saída. **Justificativa:** os casos críticos ficam deterministas, auditáveis e sem custo de inferência.

5. Divisão da equipe

Integrante	RM	Participação efetiva
Arthur Maziviero Faria	573928	Pendente de confirmação
Jun Uehara	570537	Pendente de confirmação
Felipe de Souza Gallo	569680	Pendente de confirmação
Roberson Reguero Luiz Junior	573031	Pendente de confirmação
Tommaso C. Nagliatti	572147	Pendente de confirmação
Matheus Martins Lacerda	570843	Pendente de confirmação

Turma: **A CONFIRMAR PELO GRUPO.** Participação individual pendente de confirmação com evidências reais.

Referências técnicas

LangGraph - Memory: <https://docs.langchain.com/oss/python/langgraph/add-memory>
LangChain - Guardrails: <https://docs.langchain.com/oss/python/langchain/guardrails>
OpenAI - Models: <https://developers.openai.com/api/docs/models>